















Layer 1 — params vs searchParams

This distinction is **super important** in Next.js App Router. And it connects directly to our static/dynamic discussion.

Let's use the same product example.

1 Two URLs that look similar

Imagine these two URLs:

`/products/123`



and:

`/products?id=123`



They both contain `123`.

But Next.js treats them differently.

URL 1 — `/products/123`

If your filesystem is:

```
app/  
└─ products/  
    └─ [id]/  
        └─ page.tsx
```



then:

`/products/123`



matches:

`/products/[id]`



Here `123` is a **route parameter**.

You access it using:

```
export default async function Page({ params }) {  
  const { id } = await params  
  
  return <h1>Product {id}</h1>  
}
```

So:

URL
/products/123
↓
[id]
↓
params.id
↓
"123"

URL 2 — /products?id=123

Now imagine:

app/
└─ products/
 └─ page.tsx

There is **no** [id] .

The URL is:

/products?id=123

Here:

/products

is the route.

And:

`?id=123`

is a **query string / search parameter**.

You access it using:

```
export default async function Page({ searchParams }) {  
  const { id } = await searchParams  
  
  return <h1>Product {id}</h1>  
}
```

So:

```
URL  
/products?id=123  
  |  
  ▼  
searchParams.id  
  |  
  ▼  
"123"
```



2 Visual comparison

```
/products/123  
  |  
  ▼  
URL PATH  
  |  
  ▼  
[id] segment  
  |  
  ▼  
params.id
```



Whereas:

```
/products?id=123  
  |  
  ▼  
QUERY STRING  
  |
```



▼
searchParams.id

🔥 That's the fundamental difference.

3 Why does Next.js care?

Because route parameters are part of the **route identity**.

For example:

```
/products/123  
/products/456  
/products/789
```



are three different route paths.

You have:

```
/products/[id]
```



as a dynamic route.

That's where:

```
generateStaticParams()
```

becomes useful.

You can say:

```
export async function generateStaticParams() {  
  return [  
    { id: "123" },  
    { id: "456" },  
    { id: "789" }  
  ]  
}
```

Then Next.js knows about those paths ahead of time:

```
/products/123  
/products/456  
/products/789
```



4 Search params don't create new filesystem routes

With:

```
/products?id=123  
/products?id=456  
/products?id=789
```



the filesystem route is still:

```
/products
```



The query string changes.

Conceptually:

```
/products?id=123  
  |  
  └─ same route: /products  
  
/products?id=456  
  |  
  └─ same route: /products  
  
/products?id=789  
  |  
  └─ same route: /products
```



So you don't create:

```
products/[id]/page.tsx
```



for query parameters.

5 Now let's connect this to static vs dynamic rendering

Here's where it gets interesting.

Suppose:

```
export default async function Page({ params }) {  
  const { id } = await params  
  
  return <h1>Product {id}</h1>  
}
```

and:

```
/products/[id]
```



Does `params.id` automatically make the route dynamically rendered?

✗ No.

This is a **dynamic route**, but it can still be statically rendered.

For example:

```
export async function generateStaticParams() {  
  return [  
    { id: "123" },  
    { id: "456" }  
  ]  
}
```

Then Next.js can generate:

```
/products/123  
/products/456
```



ahead of time.

So:

Dynamic Route

↓

does NOT automatically mean

↓

Dynamic Rendering



We've already established this. 🔥

6 Now compare searchParams

Suppose:

```
export default async function Page({ searchParams }) {  
  const { id } = await searchParams  
  
  return <h1>Product {id}</h1>  
}
```

The request could be:

/products?id=123



or:

/products?id=456



The value comes from the **incoming request URL**.

Under our traditional Layer-1 model, this is request-specific information.

So:

```
searchParams  
  ↓  
request-specific  
  ↓  
dynamic rendering
```



This is different from a statically generated:

/products/[id]



where `generateStaticParams()` gives Next.js known values ahead of time.

7 This gives us a really useful comparison

/products/123

```
filesystem:  
products/[id]/page.tsx
```



```
123
```

```
↓
```

```
params.id
```

Can potentially be pre-rendered.

```
/products?id=123
```

```
filesystem:  
products/page.tsx
```



```
123
```

```
↓
```

```
searchParams.id
```

Under the traditional model, using `searchParams` makes the rendering request-dependent.

8 Why can't `generateStaticParams()` solve search params?

Because `generateStaticParams()` is designed for:

```
Dynamic route segments
```



such as:

```
/[id]  
/[slug]  
/[username]
```



It returns:

```
[  
  { id: "123" },  
  { id: "456" }  
]
```

It doesn't say:

```
[  
  { searchParams: { id: "123" } }  
]
```

That's not what it's for.

9 Think about route identity

This is the easiest mental model.

Dynamic segment

```
/products/[id]
```



The `id` is part of the **pathname**.

```
/products/123
```

↑
pathname



Search parameter

```
/products?id=123
```

↑
query string



So:

```
PATH
```

↓
params

```
QUERY STRING
```

↓
searchParams



🔥 10 One subtle correction to our earlier simplified rule

We've been saying:

```
"searchParams" → dynamic rendering
```

That's a good **traditional Layer-1 mental model**, but don't treat it as a universal statement about every current Next.js architecture.

Modern Next.js has changed how request-time data can be isolated using **Cache Components / PPR-style behavior**.

So later we'll revisit:

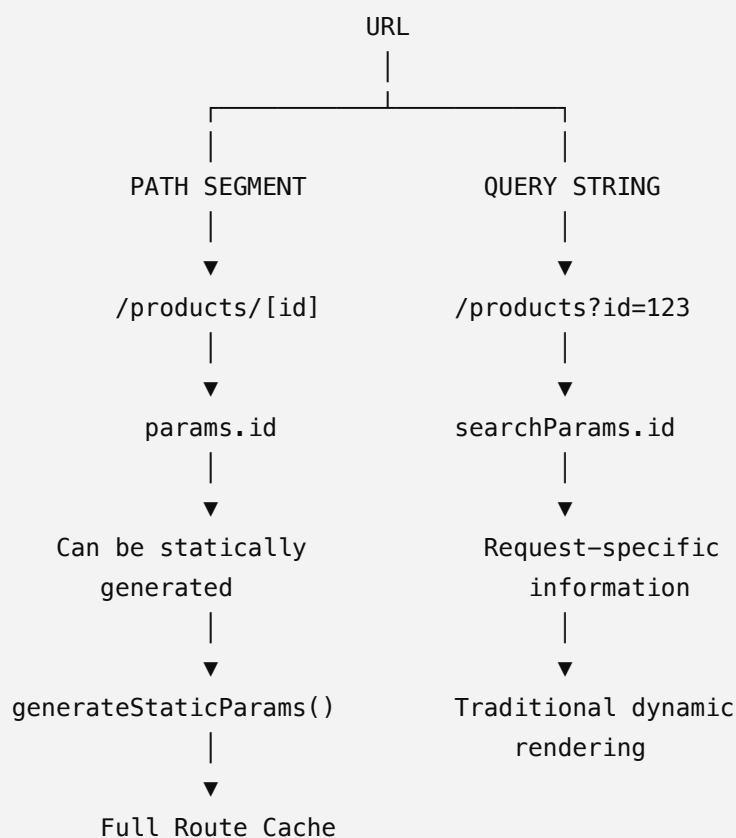
```
searchParams
cookies()
headers()
```



under Layer 2 and see how they can affect **only part of the rendered output**.

For now, we're deliberately learning the simpler traditional model.

Your complete picture now



One interview question for you

Suppose we have:

`app/products/[id]/page.tsx`

and:

```
export async function generateStaticParams() {
  return [
    { id: "1" },
    { id: "2" },
    { id: "3" }
  ]
}

export default async function Page({ params }) {
  const { id } = await params

  return <h1>Product {id}</h1>
}
```

Then users request:

```
/products/1
/products/2
/products/999
```



Before we move on, the thing I want you to understand is **exactly what happens to each of those three requests**—especially `/products/999`.

That's the next step: `generateStaticParams()` + `dynamicParams` + **Full Route Cache**, which completes the dynamic-route side of Layer 1.



ok



`generateStaticParams()` + `dynamicParams` + **Full Route Cache**

Let's finish this piece properly. This is where **dynamic route $\neq$ dynamic rendering** becomes crystal clear.

We'll use:

```
app/  
└─ products/  
    └─ [id]/  
        └─ page.tsx
```



and:

```
</> TypeScript  
  
export async function generateStaticParams() {  
  return [  
    { id: "1" },  
    { id: "2" },  
    { id: "3" }  
  ]  
}  
  
export default async function Page({ params }) {  
  const { id } = await params  
  
  return <h1>Product {id}</h1>  
}
```



1 What does `generateStaticParams()` actually do?

Think of `[id]` as a **template**:

```
/products/[id]
```



And:

```
</> TypeScript  
  
generateStaticParams()
```



returns:

```
id = 1  
id = 2  
id = 3
```



So Next.js can determine:

```
/products/1  
/products/2
```



/products/3

These are the paths it knows about ahead of time.

Conceptually during build:

generateStaticParams()



```
1
2
3
```



render each



```
/products/1
/products/2
/products/3
```



Full Route Cache



2 What is actually stored?

Let's pretend:

/products/1



produces:

HTML



```
<h1>Product 1</h1>
```

and:

/products/2



produces:

HTML



```
<h1>Product 2</h1>
```

and:

```
/products/3
```



produces:

HTML



```
<h1>Product 3</h1>
```

Then conceptually:

FULL ROUTE CACHE



```
/products/1
```

↓

HTML + RSC Payload
for Product 1

```
/products/2
```

↓

HTML + RSC Payload
for Product 2

```
/products/3
```

↓

HTML + RSC Payload
for Product 3

Again, **the** `[id]` **component code** isn't what gets stored.

The component is executed separately for each parameter value.

3 Now `/products/1`

User requests:

```
/products/1
```



Next.js can find:

Full Route Cache



```
|  
└─ /products/1
```

So conceptually:

Browser



Next.js



Full Route Cache HIT



rendered result



HTTP response



Browser



No need to render the page again.

4 /products/2

Same thing:

Browser



/products/2



Full Route Cache HIT



HTML + RSC result



Browser



5 Now the interesting one: /products/999

We never returned:

```
</> TypeScript
```

```
{ id: "999" }
```



from:

```
</> TypeScript
```

```
generateStaticParams()
```



So:

/products/999



wasn't pre-rendered during the build.

Now what happens?

It depends on `dynamicParams` .

This is the key.

6 Case A — `dynamicParams = false`

Suppose we add:

</> TypeScript



```
export const dynamicParams = false
```

So:

</> TypeScript



```
export async function generateStaticParams() {  
  return [  
    { id: "1" },  
    { id: "2" },  
    { id: "3" }  
  ]  
}  
  
export const dynamicParams = false
```

Now Next.js treats the generated params as the allowed set.

So:

```
/products/1    → ✓  
/products/2    → ✓  
/products/3    → ✓  
/products/999 → ✗
```



The request for `/products/999` results in:

404



Conceptually:

Request /products/999

↓

Is 999 in generated params?

↓

NO

↓

dynamicParams = false

↓

404



7 Case B — `dynamicParams = true`

Now suppose:

`</> TypeScript`

```
export const dynamicParams = true
```



(or you're using the default behavior that allows ungenerated params).

We have:

Known at build:

1
2
3



But the user requests:

999



Next.js can handle that route rather than automatically returning 404.

Conceptually:

Request /products/999

↓

Not in Full Route Cache

↓

Next.js handles route

↓

render /products/999

↓

response



Depending on the route's rendering/caching configuration, that result can subsequently participate in the applicable caching behavior.

The important thing is:

Not being returned by `generateStaticParams()` does not automatically mean "404".

`dynamicParams` controls that behavior.

8 Why is this useful?

Imagine Amazon has:

`/products/[id]`



and:

10 million products



You obviously don't necessarily want:

next build

↓

generate 10 million pages



Instead you could pre-render a selected set:

Popular products:

1
2
3
4
5



Then other products can be handled when requested.

Conceptually:

Build

|

├─ `/products/1`

├─ `/products/2`



```
|— /products/3  
|  
|— ...
```

Later request:

```
/products/999999  
  ↓  
handled according to dynamicParams  
and the route's rendering/caching behavior
```

This is one reason dynamic routes are so powerful.

9 Here's the terminology trap again

We now have:

```
/products/[id]
```



This is a:

Dynamic Route

because `[id]` is a dynamic path segment.

But:

```
/products/1
```



could have been:

Statically generated

because Next.js knew:

```
id = 1
```



from `generateStaticParams()`.

So:

```
Dynamic Route  
  ≠  
Dynamic Rendering
```



Keep that distinction permanently.

10 Now let's connect this to Full Route Cache

Suppose:

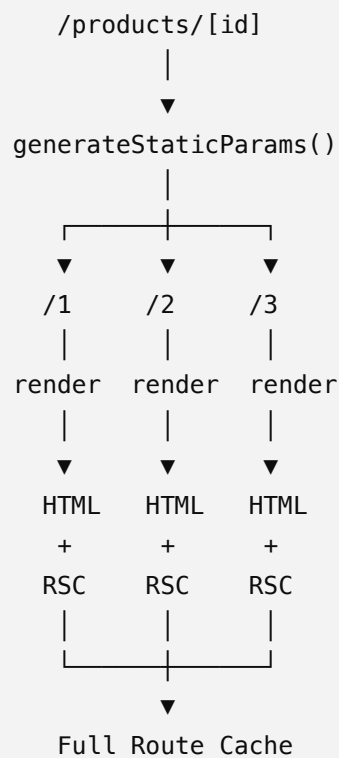
```
</> TypeScript
```

```
generateStaticParams()
```

returns:

```
1  
2  
3
```

Then during build:



That's the big picture.

11 Now let's introduce API data

Realistically, your page won't just do:

```
</> TypeScript
```

```
<h1>Product {id}</h1>
```

It'll probably do:

```
</> TypeScript
export default async function Page({ params }) {
  const { id } = await params

  const response = await fetch(
    `https://api.example.com/products/${id}`
  )

  const product = await response.json()

  return (
    <div>
      <h1>{product.name}</h1>
      <p>{product.price}</p>
    </div>
  )
}
```

Now the pipeline becomes:

```
generateStaticParams()
  ↓
id = 1, 2, 3
  ↓
render /products/1
  ↓
fetch /products/1
  ↓
API data
  ↓
render HTML + RSC
  ↓
Full Route Cache
```

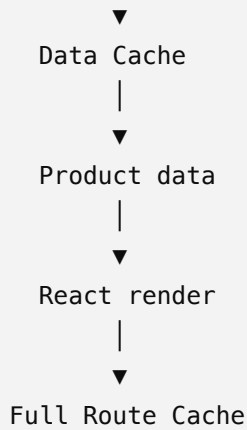
And separately:

```
API response
  ↓
Data Cache
```

if the fetch is configured/cacheable accordingly.

So:

```
API
|
```



🧠 12 One subtle thing about `generateStaticParams()`

This function is primarily about:

Which dynamic path parameters should be generated ahead of time?

It is **not** a general "make this route static" switch.

For example:

`</> TypeScript`

`generateStaticParams()`

could give:

1
2
3

but if your page also does:

`</> TypeScript`

`cookies()`

you've introduced request-specific information.

So don't think:

`generateStaticParams()`

↓

`"STATIC!!!"`

Instead:

```
generateStaticParams()
```

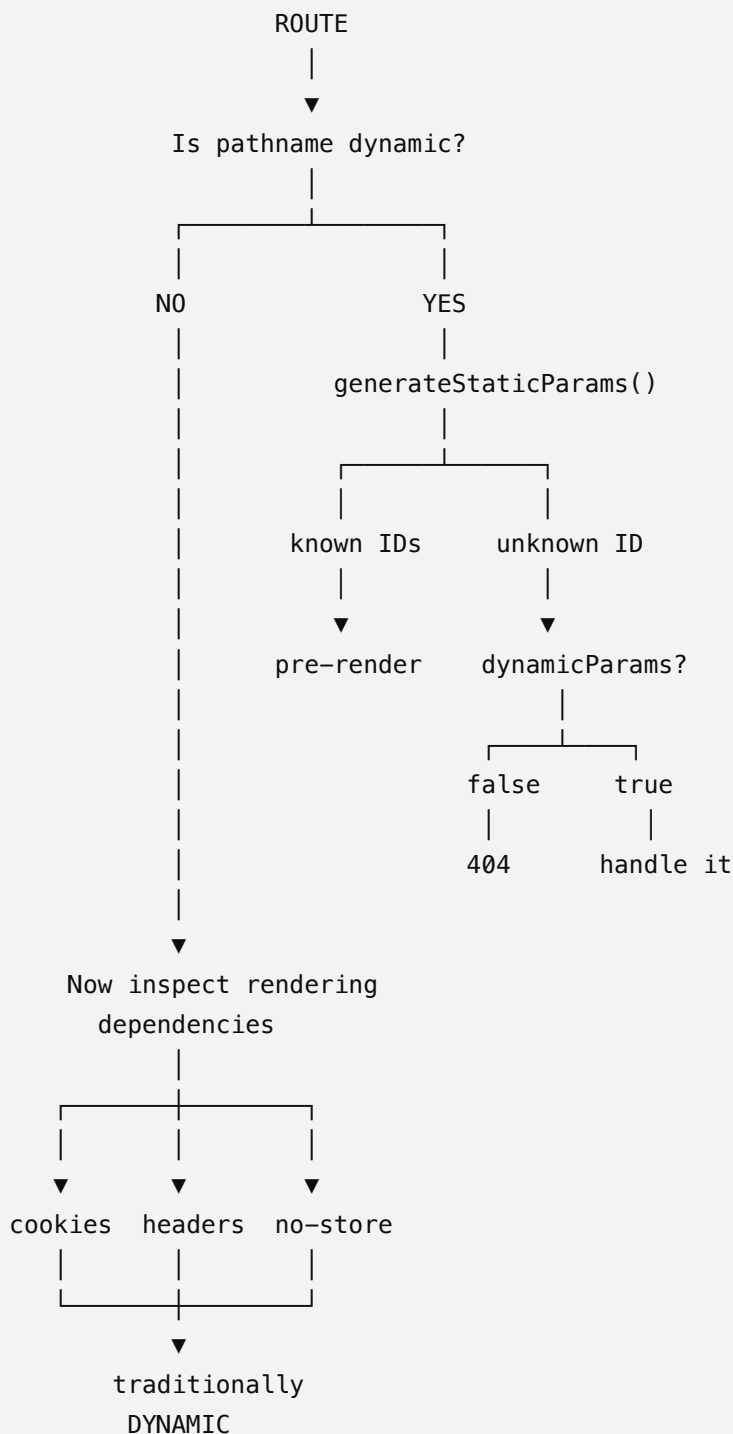
↓

```
"These parameter values are known ahead of time."
```



Then Next.js still has to consider the route's other rendering dependencies.

🎯 Our Layer-1 decision tree is getting complete





Where we are now

We've covered:

- ✓ Static route
- ✓ Dynamic route
- ✓ Cacheable fetch
- ✓ no-store
- ✓ SSG
- ✓ SSR
- ✓ ISR
- ✓ params
- ✓ searchParams
- ✓ generateStaticParams()
- ✓ dynamicParams
- ✓ Full Route Cache



The **last major pieces of Layer 1** are:

```
cookies()  
headers()
```



and then we need to connect **all of them into one actual request/rendering decision tree**.

After that, we'll do a **big end-to-end exercise** where I give you several real Next.js pages and you tell me:

```
Static or Dynamic?  
SSG / SSR / ISR?  
Data Cache?  
Full Route Cache?  
What happens at request time?
```



Then I'll correct your reasoning. That's the point where this stuff should really stick. 🧠





Ahhh, yes — I see the exact confusion now. 😊 You're asking:

"If `/products` is cached and doesn't depend on the cookie, and I change a cookie in the browser, how does the server ever learn that the cookie changed?"

The answer is: **it learns it on the next request that actually reaches the server.** And if that request is satisfied entirely by a CDN cache, the origin server may **not need to know at all.**

Let's separate these cases.

1. Cookie changes in the browser

Suppose:

Browser

`theme = light`



Server Action changes it:

```
cookies().set("theme", "dark")
```



Response:

```
</> http
```

```
Set-Cookie: theme=dark
```



Browser now has:

Browser

theme = dark



At this point:

Server: ❌ hasn't received "theme=dark" as a new request

Browser: ✅ knows theme=dark



2. Then user requests `/products`

Browser makes:

```
</> http
```

```
GET /products
```

```
Cookie: theme=dark
```



Now there are two possibilities.

Case A — CDN handles it

Browser

| GET /products

| Cookie: theme=dark



CDN

| Cache HIT



Cached /products response



The request **doesn't reach Next.js origin**.

Therefore:

Next.js server

❌ doesn't learn about this request

❌ doesn't need to



The CDN has already answered the request.

And that's perfectly fine if `/products` doesn't care about `theme`.

3. So when does Next.js actually learn the new cookie?

When there is a request that reaches Next.js.

For example:



```
graph TD; Browser -- "Cookie: theme=dark" --> CDN; CDN -- "Cache MISS" --> Nextjs[Next.js]; Nextjs --> cookies[cookies()];
```

Now Next.js can see:

theme=dark

So:



```
graph TD; A[Cookie changed] --> B[Browser stores new cookie]; B --> C[Future HTTP request]; C --> D[Cookie automatically attached]; D --> E[If request reaches server]; E --> F[Server can read new cookie];
```

4. But here's the important realization 💡

You don't actually need to "tell the server that the cookie changed" as a separate operation.

The cookie itself is the state.

The browser effectively says:

"From now on, whenever I make an applicable request, I'll send `theme=dark`."

So imagine:

10:00

Browser:
theme=light



Server Action:

Set-Cookie: theme=dark



Browser:

theme=dark



Then:

10:05
GET /profile
Cookie: theme=dark



Server sees it.

There was no separate:

UPDATE_COOKIE /api/notify-server



required.

5. What if `/products` is completely cached?

Suppose:

`/products`



doesn't care about cookies.

CDN has:

`/products` → Product HTML



User requests it:

```
Browser
Cookie: theme=dark
  |
  ▼
CDN
  |
  ▼
Cached Product HTML
```



Next.js doesn't learn about the cookie.

And that's okay.

Because the page doesn't use it.

It's exactly like sending someone a letter that says:

```
"Here is my favorite color: dark"
```



but the person only needs to answer:

```
"What products are available?"
```



They don't need to process your favorite color. 😊

6. Now compare with `/profile`

Suppose `/profile` says:

```
</> TypeScript

const cookieStore = await cookies()
const theme = cookieStore.get("theme")?.value
```



Now the cookie matters.

Browser:

```
</> http

GET /profile
Cookie: theme=dark
```



If CDN cannot safely serve a generic cached response:

```
Browser
  ↓
CDN
  ↓
Next.js
  ↓
cookies()
  ↓
theme = dark
  ↓
render
```



Now Next.js **does** learn about it.

7. There's one more scenario: you WANT the server to know immediately

Suppose you have some client-side cookie that you change using JavaScript:

```
</> JavaScript
```

```
document.cookie = "theme=dark"
```



The server isn't immediately notified.

If you specifically need the server to know immediately, you make a request:

```
</> JavaScript
```

```
document.cookie = "theme=dark"
```

```
await fetch("/api/theme", {
  method: "POST"
})
```



Now:

```
Browser
|
| Cookie already changed
| Cookie: theme=dark
▼
POST /api/theme
▼
```



Next.js



Server knows theme=dark

But normally you don't need this extra request unless your application has a reason to synchronize server-side state.

This is the BIG distinction

There are actually **three events**:

Event 1 — Cookie changes

Server → Set-Cookie → Browser



or JavaScript changes it.

Event 2 — Browser makes a request

Browser → HTTP Request + Cookie → network



Event 3 — Somebody processes that request

Could be:

Browser



CDN



response



or

Browser



CDN



Next.js



If CDN answers it:

Next.js doesn't see it.



If CDN forwards it:

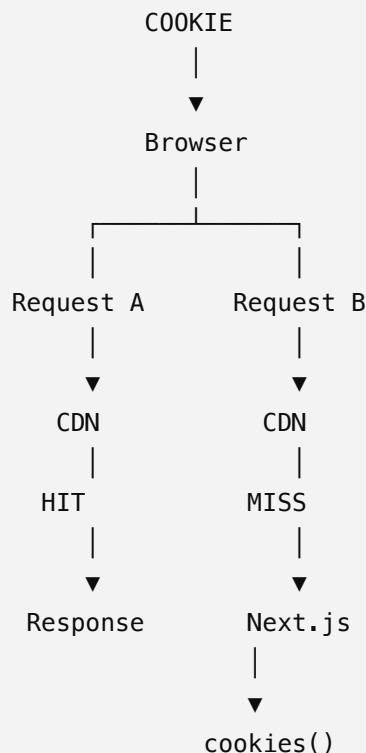
Next.js sees the cookie.



🔥 The sentence that should make this click

A cookie is not a live connection between the browser and server. It's client-side state that gets included in applicable HTTP requests.

So:



In **Request A**, Next.js never knows the cookie existed.

In **Request B**, Next.js sees the cookie.

And that's completely normal. 👍

The next thing that will make this whole topic click is understanding **what exactly a CDN cache key is and whether cookies are part of that key**—because that determines whether two users with different cookies can receive the same cached response.



ok

Perfect 😊 Let's lock this piece in.

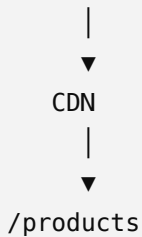
🧠 Next: CDN Cache Key + Cookies

The question we'll answer is:

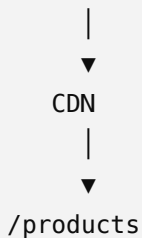
If two users request `/products` but have different cookies, how does the CDN know whether it can give them the same cached response?

We'll trace:

User A
Cookie: theme=dark



User B
Cookie: theme=light



Then we'll understand:

1. What a cache key actually is
2. Why `/products` normally maps to one cached response
3. How `Vary` works
4. Whether cookies can be part of cache variation
5. Why personalized pages generally shouldn't be blindly CDN-cached
6. How this differs from **Next.js Data Cache / Full Route Cache / Router Cache**
7. Finally, we'll return to our **Layer 1 static vs dynamic exercise**. 🚀



ok

Alright 😊 let's do it. This is one of those concepts that makes **CDN caching + cookies** suddenly click.

🌐 **CDN Cache Key — What actually identifies a cached response?**

Imagine the CDN receives:

```
</> http
GET /products
```



The CDN needs to answer:

"Do I already have a response for this request?"

It uses something called a **cache key**.

A simplified cache key might be:

```
https://example.com/products
```



So:

```
Request
GET /products
  ↓
Cache Key
example.com/products
  ↓
CDN Cache
  ↓
Cached Response
```



1. Two users request the same URL

Suppose:

User A

```
GET /products
Cookie: theme=dark
```



User B

GET /products
Cookie: theme=light

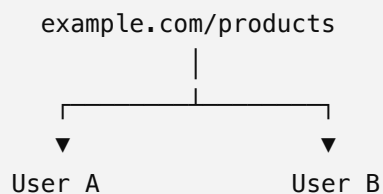


If the CDN's cache key is simply:

example.com/products



then both requests map to:



So the CDN could potentially return the **same cached response** to both.

That's perfectly fine **if the response doesn't depend on the cookie.**

2. Example where this is perfectly safe

Your `/products` page:

`</>` TypeScript



```
export default async function ProductsPage() {  
  const products = await getProducts()  
  
  return (  
    <ul>  
      {products.map(product => (  
        <li key={product.id}>{product.name}</li>  
      ))}  
    </ul>  
  )  
}
```

The page doesn't care about:

theme
session
language
user ID



So:

```
User A
Cookie: theme=dark
  |
  ▼
/products
  |
  ▼
  CDN
  |
  ▼
Cached products response
```



User B:

```
User B
Cookie: theme=light
  |
  ▼
/products
  |
  ▼
  CDN
  |
  ▼
SAME cached products response
```



That's fine.

Because:

```
Cookie —X—> Products HTML
```



The cookie doesn't influence the response.

3. Now let's make the cookie matter

Suppose:

```
</> TypeScript

export default async function ProductsPage() {
  const cookieStore = await cookies()

  const theme = cookieStore.get("theme")?.value
```



```
return (  
  <div className={theme}>  
    Products  
  </div>  
)  
}
```

Now:

Cookie
↓
theme
↓
HTML



User A:

theme=dark
↓
<div class="dark">



User B:

theme=light
↓
<div class="light">



So one cached response cannot represent both.

4. Here's the problem with a naive CDN

Suppose CDN already has:

Cache:

/products
↓
<div class="dark">



User B arrives:

```
</> http  
  
GET /products  
Cookie: theme=light
```



If CDN says:

"Oh, `/products` is cached! I'll return it."

then:

```
User B
theme=light

↓

CDN

↓

<div class="dark">
```

🔥 Wrong response.

That's a **cache correctness problem**.

5. So how does a CDN handle this?

There are several mechanisms in real systems.

One important HTTP mechanism is:

```
</> http
Vary
```

For example:

```
</> http
Vary: Accept-Language
```

This tells caches:

"The response varies based on this request header."

So conceptually:

```
/products + language=en
/products + language=fr
```

can be treated as different variants.

6. What about cookies?

Here's where things get more nuanced.

You *can* design caching behavior around cookies, but you generally don't want to casually make:

```
Cache Key =  
URL + entire Cookie header
```



because imagine:

```
Cookie:  
session=abc123;  
theme=dark;  
cart=92837;  
...
```



Now every user's session could produce a completely different cache entry.

You'd destroy much of the benefit of shared caching.

You could theoretically get:

```
/products + userA cookie  
/products + userB cookie  
/products + userC cookie  
/products + userD cookie  
...
```



That's basically **personalized caching**.

7. This is why personalized pages are tricky

Imagine:

```
<> TypeScript  
  
const session = cookies().get("session")  
  
return <h1>Welcome, {sessionUser.name}</h1>
```



User A:

Welcome Gopi



User B:

Welcome Rahul



You absolutely cannot have:

/products



Welcome Gopi



as one generic shared CDN response.

Otherwise:

User B



CDN



"Welcome Gopi"



😞 That's obviously terrible.

So systems generally avoid treating such personalized responses as ordinary shared public cache entries, or explicitly vary/cache them according to an appropriate strategy.

8. Now connect this to Next.js

This is why our earlier statement was:

cookies()



request-dependent



dynamic rendering



Under the **traditional Layer 1 model**, Next.js doesn't say:

"I'll just put this personalized page in the generic Full Route Cache."

Instead, the route is treated as request-dependent.

Conceptually:

```
Request
  ↓
Next.js
  ↓
cookies()
  ↓
render THIS request
  ↓
response
```



9. But here's a very important distinction

Don't think:

"If a request contains a cookie, CDN can never cache it."

That's too simplistic.

The correct idea is:

The presence of a cookie doesn't automatically make every response uncacheable. What matters is whether the response is safe to reuse for another request.

For example:

Cookie: theme=dark



but `/products` doesn't use theme.

The response could still potentially be shared.

Whereas:

Cookie: userId=123



and the response says:

Welcome Gopi



That's personalized.

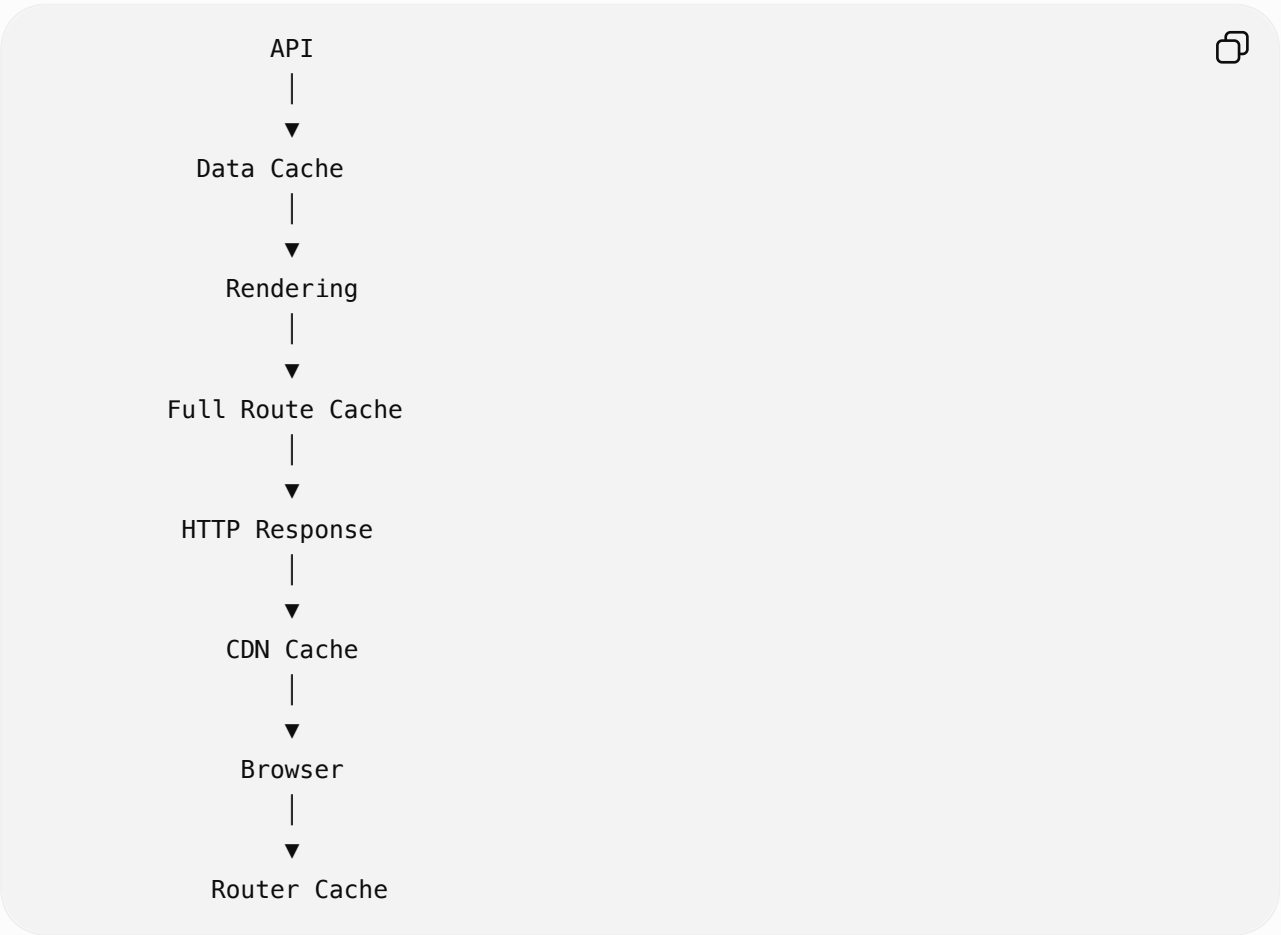
Different situation.

10. Now our three cache layers become clearer

Let's put them side-by-side.

Layer	What it caches	Where
Data Cache	Fetches data	Next.js/server infrastructure
Full Route Cache	Rendered route result	Next.js/server infrastructure
CDN/HTTP cache	HTTP responses	CDN/edge/intermediary
Router Cache	RSC payload	Browser/client

So:



And importantly, **these aren't duplicate copies of exactly the same thing for the same purpose.**

11. One killer example 🔥

Imagine your page:

</> TypeScript



```
export default async function Page() {
  const cookieStore = await cookies()
  const theme = cookieStore.get("theme")?.value

  const response = await fetch(
    "https://api.example.com/products",
    {
      next: { revalidate: 60 }
    }
  )

  const products = await response.json()

  return (
    <div className={theme}>
      ...
    </div>
  )
}
```

Now there are **two different kinds of information**:

Cookie

theme=dark



Request-specific.

Products API data

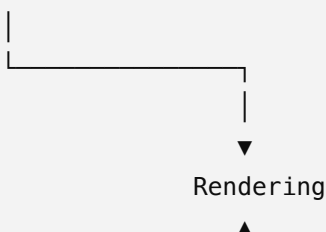
iPhone
MacBook
AirPods

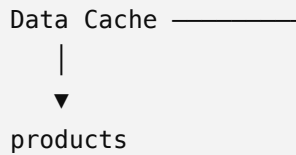


Potentially cacheable/reusable.

So conceptually:

Cookie





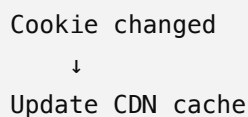
This is a really important insight:

One route can have request-specific information AND separately cacheable data dependencies.

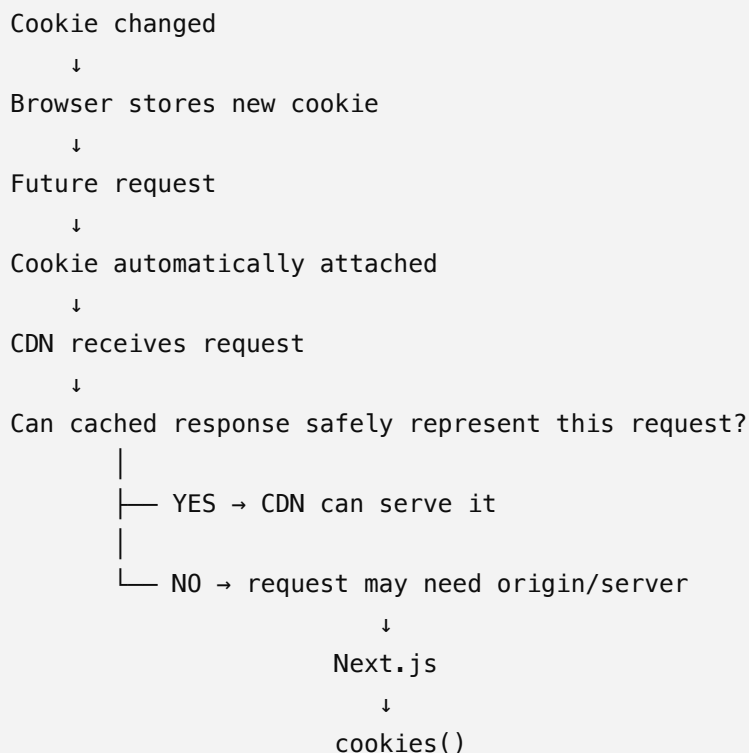
And this becomes **hugely important when we get to Layer 2 / Cache Components**, because modern Next.js gives you more precise control over which parts are cached and which parts remain request-dependent.

Your mental model now

Don't think:



Instead:



That's the whole game. 🎯

And now we're ready for the **Layer 1 debugging exercise**. I'll give you a route and code, and you tell me what happens. I'll deliberately mix **dynamic URL segments**, **cookies**, `searchParams`, `fetch`, `no-store`, **and** `revalidate` so you have to distinguish all of them.



ChatGPT can make mistakes. Check important info.